DEPARTMENT OF ENGINEERING MATHEMATICS

Edge-Deployable Rock Detection and Burial Status Classification

Tanishk Nanasaheb Shinde
Megh Kashilkar
Sanskar Sanju Gade
Jitendra Suwalka

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree of Master of Science in the Faculty of Engineering.

11th September, 2026

Supervisor: Dr. Felipe Campelo
Assistant Supervisors: Dr. Ahmad Al-Mallahi
(Dalhousie University / McCain Foods representative) and Jason McLaren

Declaration

This dissertation is submitted to the University of Bristol in accordance with the requirements of the degree of MSc in the Faculty of Engineering. It has not been submitted for any other degree or diploma of any examining body. Except where specifically acknowledged, it is all the work of the Authors.

Tanishk Nanasaheb Shinde
Megh Kashilkar
Sanskar Sanju Gade
Jitendra Suwalka

11th September, 2026

Contents

List of Figures

List of Tables

Abstract

Agricultural rocks can interfere with field operations and potentially damage machinery, motivating computer-vision systems capable of detecting their location and burial status. This dissertation investigates the deployment of a two-class YOLOv8n detector for identifying completely exposed and half-buried rocks in agricultural RGB imagery on a Raspberry Pi 5. The work focuses on the transition from model development to resource-constrained edge inference rather than implementing the wider GPS mapping or automated rock-removal system.

The original dataset contained 448 images and 706 annotated rocks. YOLOv8n was selected from YOLOv8n, YOLO11n and YOLO11s using validation mAP50–95 as the predefined primary selection criterion. The selected detector was then prepared in six deployment representations: PyTorch FP32, ONNX FP32, TFLite FP32, TFLite INT8, NCNN FP32 and NCNN FP16. A common 90-image, 135-rock, 640×640 workload was used to evaluate detection quality, inference latency, throughput, CPU utilisation, process memory, rock-count error, temperature and operational behaviour on Raspberry Pi 5 CPU hardware.

Five representations completed the full benchmark. PyTorch FP32 achieved the strongest aggregate edge-benchmark detection quality, with mAP50 of 0.548, but required 397.48 ms mean latency and achieved only 2.52 median FPS. TFLite INT8 reduced mean latency to 44.13 ms and increased median throughput to 22.62 FPS, while mAP50 decreased to 0.503. NCNN FP16 provided an alternative accuracy–efficiency compromise, achieving 13.82 median FPS with mAP50 of 0.537. ONNX FP32 supported single-image inference but did not complete sustained benchmarking.

TFLite INT8 additionally completed approximately 2,533 continuous inferences over two minutes without recorded throttling or reboot, although temperature increased to approximately 76–77 °C. Half-buried rocks remained substantially more difficult to recognise than completely exposed rocks, and this weakness existed before edge conversion.

The results support the hypothesis that edge-optimised and reduced-precision representations can substantially improve Raspberry Pi inference efficiency while introducing runtime-dependent accuracy and operational trade-offs. TFLite INT8 was therefore selected as the preferred representation for the tested CPU-only configuration. The resulting system constitutes an academic edge-deployment proof-of-concept rather than a production-ready agricultural solution.

Supporting Technologies

The principal third-party hardware and software resources used in the project were:

- **Raspberry Pi 5 Model B** — target edge-computing platform for CPU-only deployment and benchmarking of the selected YOLOv8n detector.
- **Official Raspberry Pi 27 W USB-C power supply (5.1 V / 5 A)** — used for the final Raspberry Pi benchmark configuration.
- **Debian GNU/Linux 13** — operating system used on the Raspberry Pi deployment platform.
- **Python 3.13.5** — principal Raspberry Pi programming environment for inference, benchmarking, verification and continuous-inference testing.
- **Ultralytics 8.4.115** — used for YOLO model handling, inference and preparation of alternative deployment representations.
- **PyTorch** — original FP32 representation and reference implementation for the selected detector.
- **ONNX / ONNX Runtime 1.28.0** — used for ONNX FP32 export and CPU inference; isolated inference succeeded but a valid sustained benchmark was not completed.
- **TensorFlow Lite / LiteRT** — used for FP32 and post-training INT8 deployment representations and on-device inference.
- **NCNN** — lightweight inference framework used for FP32 and FP16 deployment on the Raspberry Pi CPU.
- **OpenCV 5.0.0** — image loading, preprocessing and supporting computer-vision operations.
- **NumPy 2.5.1** — numerical and array-based processing in inference and evaluation scripts.
- **Pandas 3.0.5** — organisation and analysis of benchmark outputs and experimental results.
- **psutil 7.2.2** — CPU-utilisation and process-memory monitoring during Raspberry Pi benchmarking.
- **CLIP ViT-B/32** — separate crop-level burial-status classification research branch; it was not part of the verified Raspberry Pi benchmark.
- **SSH and SCP** — remote Raspberry Pi access and transfer of deployment files and benchmark outputs.

Notation and Acronyms

Important acronyms and notation used throughout the dissertation are summarised below. Terms are also defined at first use in the main text.

Acronyms and technical abbreviations

AI	Artificial Intelligence
AP	Average Precision
CLIP	Contrastive Language–Image Pre-training
CPU	Central Processing Unit
F1	Harmonic mean of precision and recall
FP16	16-bit floating-point numerical representation
FP32	32-bit floating-point numerical representation
FPS	Frames Per Second
GPS	Global Positioning System
INT8	8-bit integer numerical representation
IoU	Intersection over Union
MAE	Mean Absolute Error
mAP	mean Average Precision
mAP50	mean Average Precision evaluated at IoU threshold 0.50
mAP50–95	mean Average Precision averaged across IoU thresholds from 0.50 to 0.95 in increments of 0.05
NCNN	Lightweight neural-network inference framework for mobile and embedded deployment
ONNX	Open Neural Network Exchange
RAM	Random Access Memory
RGB	Red–Green–Blue image representation
RSS	Resident Set Size; process memory resident in physical RAM
SCP	Secure Copy Protocol
SPPF	Spatial Pyramid Pooling — Fast
SSH	Secure Shell
TFLite	TensorFlow Lite
LiteRT	Google's on-device machine-learning runtime ecosystem used for TensorFlow Lite model execution
YOLO	You Only Look Once

Mathematical notation

B_{pred}	Predicted bounding box
B_{gt}	Ground-truth bounding box
TP	Number of true-positive detections
FP	Number of false-positive detections
FN	Number of false-negative detections
y_i	Ground-truth rock count for image i
$\hat{y}_i$	Predicted rock count for image i
N	Number of images included in an evaluation
W, H	Width and height of the original image
W', H'	Width and height after aspect-ratio-preserving resizing
s	Letterbox image-scaling factor
r	Real-valued quantity before quantisation
q	Quantised integer representation
S	Quantisation scale factor
Z	Quantisation zero point

Acknowledgements

The authors gratefully acknowledge Dr. Felipe Campelo for his academic supervision, regular guidance and constructive feedback throughout the project. His support helped the group refine the research question, maintain a clear distinction between completed evidence and planned work, and present the detector-development and edge-deployment results with the appropriate level of critical evaluation.

The authors also thank the Assistant Supervisors, Dr. Ahmad Al-Mallahi and Jason McLaren, for helping to situate the work within the wider agricultural-engineering challenge. The Dalhousie University / McCain Foods project context provided the practical motivation for investigating field-rock detection, burial-status classification and the constraints that arise when a computer-vision model must ultimately operate close to the point of data capture.

The group acknowledges the University of Bristol Department of Engineering Mathematics and the MSc Data Science programme for the academic environment, computing access and supporting resources used during the project. Finally, the authors recognise the collaborative effort of all four group members across dataset preparation, model development, conversion, Raspberry Pi benchmarking, evidence checking, analysis and report preparation. Open sharing of experimental outputs and careful cross-checking of results were essential to producing a coherent collective submission.

Chapter 1

Introduction

1.1 Project Context and Motivation

This dissertation reports the outcomes of a four-member group project undertaken by Tanishk Nanasaheb Shinde, Megh Kashilkar, Sanskar Sanju Gade and Jitendra Suwalka. Unless explicitly stated otherwise, the methods, experimental results and conclusions presented in Chapters 1–5 describe the collective project work rather than the individual contribution of any one group member.

Precision agriculture increasingly combines sensing, computer vision and local computation to support field operations with more timely and spatially specific information. Object detection is particularly relevant because it can transform ordinary imagery into structured information about the position, class and number of objects in a scene. Reviews of precision- farming vision systems show both the rapid uptake of object detection and the continuing importance of dataset quality, environmental variability and deployment constraints [1, 2].

The wider project motivating this study concerns rocks in potato fields. Rocks can obstruct field operations, contribute to equipment wear and create substantial manual removal work. The Dalhousie University / McCain Foods project envisages detecting rocks during normal field operations, associating detections with position information and ultimately supporting automated removal [4, 3]. The group project deliberately addresses a narrower and verifiable part of that wider vision: detecting rocks in RGB images, distinguishing between completely exposed and half-buried rocks, and determining whether the detector can be run effectively on low-cost edge hardware.

1.2 Problem Definition and Edge-Deployment Challenge

The available dataset contains 448 RGB images and 706 labelled rock instances. The two target classes are *rock completely exposed* and *rock half buried*. This is not only a localisation task. Burial status matters because a partially visible rock has less visual evidence, may share colour and texture with the surrounding soil, and is more affected by occlusion. Related agricultural vision literature similarly identifies background complexity, changing illumination, occlusion and limited labelled data as recurring obstacles to robust field detection [1, 22].

A detector that performs acceptably during development is not automatically suitable for an embedded device. The deployment representation changes the runtime, numerical precision, memory behaviour and available optimisations even when the underlying trained model is nominally the same. The Raspberry Pi 5 therefore creates a system-level engineering question: how much accuracy is retained when latency is reduced, what resource profile accompanies each runtime, and can the configuration operate reliably under repeated inference?

The project compared three lightweight candidate detectors—YOLOv8n, YOLO11n and YOLO11s—and selected YOLOv8n using validation mAP50–95 as the predefined primary criterion. The selected model was then represented in six edge formats: PyTorch FP32, ONNX FP32, TFLite FP32, TFLite INT8, NCNN FP32 and NCNN FP16. The study consequently separates two decisions that can otherwise be conflated: *detector architecture selection* based on validation results, and *runtime representation selection* based on Raspberry Pi behaviour.

1.3 Aim, Research Question and Hypothesis

The principal aim is to investigate the feasibility and practical trade-offs of deploying a two-class YOLOv8n agricultural rock detector on a Raspberry Pi 5 using alternative edge-runtime representations.

The primary research question is:

> *How effectively can a two-class YOLOv8n rock detector be deployed on a Raspberry Pi 5, and what accuracy, latency, memory and operational trade-offs arise across alternative edge-runtime representations?*

The research hypothesis is that reduced-precision and edge-optimised representations will substantially reduce Raspberry Pi inference latency relative to PyTorch FP32, but the magnitude of improvement will vary by runtime and may involve losses in detection accuracy or operational robustness.

1.4 Objectives and Project Contribution

The project objectives are to:

1. establish the two-class detector and evaluation context;
2. prepare six alternative edge-runtime representations of the selected detector;
3. create a reproducible Raspberry Pi evaluation workflow;
4. benchmark detection quality, latency, throughput and resource behaviour under a common workload;
5. investigate operational reliability and document runtime failures or negative results; and
6. select and justify a preferred deployment representation for the tested Raspberry Pi configuration.

The principal contribution is therefore not a new detection architecture. It is an evidence-based translation of a trained agricultural detector from model-development conditions to constrained hardware, with the runtime alternatives compared using the same images and a common set of accuracy, speed, counting, resource and operational measures.

1.5 Scope and Boundaries

The implemented scope includes dataset auditing, training-only augmentation, lightweight YOLO comparison, selection of YOLOv8n, model conversion, stored-image inference on a Raspberry Pi 5, rock counting from detections, and cross-runtime benchmarking. A separate CLIP ViT-B/32 crop-classification branch was explored during model development, but it was not part of the verified Raspberry Pi benchmark [14].

The study does *not* claim implementation of live-camera operation, GPS integration, verified rock size or weight estimation, depth or thermal sensing, robotic manipulation, or the complete field system described in the wider project. The Raspberry Pi evaluation is therefore an academic CPU-only proof of concept. This boundary is important because the conclusions concern the feasibility of one detection component, not production readiness of an autonomous agricultural machine.

1.6 Dissertation Organisation

Chapter 2 reviews the technical and agricultural background. Chapter 3 describes the executed workflow, from dataset preparation and detector selection to Raspberry Pi conversion and benchmarking. Chapter 4 critically evaluates model generalisation, class-specific weaknesses, accuracy–efficiency trade-offs, resource behaviour and runtime stability. Chapter 5 answers the research question, assesses the hypothesis and objectives, and prioritises the work required to move from the present proof of concept toward a more realistic field system.

Chapter 2

Background and Related Work

2.1 Precision Agriculture and Agricultural Stone Detection

Precision agriculture applies sensing, computation and data-driven decision making at a spatial and temporal resolution finer than conventional whole-field management. Computer vision is one route to this objective because cameras provide a relatively inexpensive source of detailed information about crops, weeds, obstacles and field conditions. Ariza-Sentís et al. review more than one hundred recent precision-farming studies and identify object detection and tracking as important enabling techniques, while also emphasising environmental variability and limited labelled datasets as persistent constraints [1]. A review focused specifically on YOLO in agriculture similarly highlights the attraction of single-stage detectors for tasks that need a practical compromise between accuracy and speed [2].

Stone detection is a useful example of why agricultural imagery remains difficult despite strong general-purpose vision models. Soil colour, clods, vegetation, shadows and uneven illumination produce visually complex backgrounds, while a partly buried stone may expose only a small or irregular region. Thürkow et al. demonstrate continuing research interest in agricultural stone detection using thermal imagery, illustrating that the problem is sufficiently challenging to motivate sensing strategies beyond conventional RGB alone [22]. The present study retains RGB imagery because it matches the supplied project data and the intended low-cost vision setting, but the class-specific analysis later shows why burial remains a significant source of error.

2.2 Object Detection and Lightweight YOLO Models

Object detection combines classification with localisation. Two-stage detectors such as Faster R-CNN first generate candidate regions and then classify/refine them, whereas one-stage detectors predict object categories and locations more directly [19]. SSD demonstrated that a single network could perform multi-scale detection without a separate proposal stage [12]. YOLO pushed the single-stage idea further by framing detection as a unified prediction problem and is now a major family of real-time detectors [18].

Modern detectors use multi-scale features because agricultural objects can occupy very different fractions of the image. Feature Pyramid Networks formalised a widely used approach for combining semantic information across scales [10]; dense detection research has also improved the representation and optimisation of bounding boxes, including distribution-based regression in Generalized Focal Loss [9]. Although implementations differ between YOLO generations, these ideas motivate the use of a compact detector that can preserve useful multi-scale features without imposing the computation of a large backbone. The exact YOLOv8/YOLO11 implementation used in this project is supplied by Ultralytics and should therefore be interpreted according to the official implementation documentation rather than as a newly proposed architecture [23].

The project compared YOLOv8n, YOLO11n and YOLO11s. The "n" variants target low parameter and computation budgets, while YOLO11s increases capacity. The comparison is deliberately empirical: the architecture used for deployment was chosen by a predefined validation criterion rather than because one generation was assumed to be universally superior.

Conceptually, the Ultralytics detectors can be separated into a backbone, a feature-fusion neck and a detection head. The backbone converts pixels into progressively more abstract feature maps; the neck combines information across resolutions; and the head produces class and bounding-box predictions. Multi-scale fusion is important in this problem because a rock may occupy a large fraction of one image

but only a small region of another. YOLOv8 uses C2f blocks and an SPPF module before multi-scale fusion, with an anchor-free decoupled detection head. The official YOLO11 implementation changes internal modules, including C3k2/C2PSA components, while retaining the broad lightweight one-stage design [23]. These details matter for model capacity and computational demand, but they were not modified by the project; the experimental question is which supplied lightweight model generalised best under the common training protocol.

At inference time, YOLO converts an input image into a set of candidate detections in a single forward pass. The backbone extracts visual features, the neck propagates and combines information at several spatial resolutions, and the decoupled head predicts bounding-box geometry and class confidence. The resulting candidates are filtered by a confidence threshold and overlapping boxes are suppressed during post-processing. This sequence produces the box coordinates, burial-status class and confidence value required for annotation, class-specific counting and structured output.

The joint treatment of localisation and classification is particularly important for the present task. A half-buried rock provides less visible shape and texture than a completely exposed rock, so the detector must first preserve enough evidence to localise the object and then assign the correct burial class. A missed detection, an inaccurate box and a burial-status confusion therefore represent different failure modes even when they affect the same aggregate score. Reporting precision, recall, F1, mAP50 and mAP50–95 gives complementary evidence about these behaviours rather than treating "rock detected" as a single binary outcome.

YOLO is consequently the central analytical component of the project rather than only a convenient preprocessing tool. The same trained detector is carried from development into each edge-runtime representation, allowing the dissertation to distinguish model-level weaknesses from changes caused by export format, numerical precision or backend execution. This controlled continuity is necessary for interpreting whether a deployment trade-off originates in the detector or in the runtime used to execute it.

The "nano" versus "small" distinction is also relevant to edge deployment. Increasing model capacity can improve representational power but does not guarantee better generalisation on a small specialised dataset, and it increases the work required per image. Comparing YOLO11n with YOLO11s therefore provides evidence about scale within one generation, while comparing YOLOv8n with YOLO11n provides evidence about architecture generation at an edge-oriented scale. The selection rule prevents these considerations from being decided by model age or parameter count alone.

2.3 Transfer Learning and Data Augmentation

Training a detector from a pretrained representation is especially valuable when the task-specific dataset is modest. Transfer learning exploits features learned from larger source datasets and then adapts them to a target problem; work by Yosinski et al. shows that learned features vary in how transferable they are but can still substantially reduce the burden of learning from scratch [24]. Large detection datasets such as COCO also provide a common pretraining and evaluation context for modern detectors [11].

The original project dataset is small enough that overfitting and narrow coverage are material risks. Offline augmentation was therefore applied to the training partition only. Six photometric transformations were used: brightness, contrast, Gaussian noise, mild blur, hue/saturation and sharpness. Applying these transformations expanded 313 original training images to 2,191 training images. These transformations create 1,878 augmented images and 3,563 repeated annotation instances; importantly, those instances are augmented copies of existing labelled rocks rather than 3,563 independently collected objects. The validation and test partitions were not augmented, preserving a more meaningful estimate of generalisation.

Agricultural scene complexity also explains why augmentation cannot by itself solve the burial problem. A half-buried rock can be partially occluded by soil and may have ambiguous boundaries, so the detector must infer class from less visible evidence than for a fully exposed rock. This is a semantic and data-distribution challenge, not merely a numerical-precision problem introduced by edge conversion.

Field imagery combines several sources of variation that are difficult to reproduce exhaustively with augmentation. Illumination changes the apparent colour and contrast of both soil and stone; wet and dry soil can have different reflectance; crop residues and emerging vegetation introduce new textures; shadows can create rock-like boundaries; and camera height or viewing angle changes the apparent scale of an object. Occlusion is especially important for burial status because the visible fraction is itself part of the class definition. A detector may therefore correctly infer that an object is a rock while still confusing whether it is exposed or half buried. This motivates reporting both localisation-oriented mAP and class-specific behaviour rather than reducing performance to a single overall score.

The class totals in the original dataset are reasonably balanced globally (362 exposed and 344 half buried), but numerical balance does not guarantee equal visual difficulty. Half-buried instances can contain fewer discriminative pixels and more ambiguous boundaries. Consequently, future data collection should be stratified not only by class count but by burial fraction, lighting, soil type, scene density and rock appearance. This is a stronger strategy than simply creating more transformed copies of the existing scenes.

2.4 Detection and Counting Metrics

For a predicted box B_{pred} and ground-truth box B_{gt}, Intersection over Union is

$$\text{IoU} = \frac{|B_{\text{pred}} \cap B_{\text{gt}}|}{|B_{\text{pred}} \cup B_{\text{gt}}|}. \tag{2.1}$$

A prediction is treated as a correct detection only when its class and overlap satisfy the chosen evaluation rules. Precision, recall and F1 are

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F1 = \frac{2PR}{P + R}. \tag{2.2}$$

Average Precision integrates the precision–recall relationship for a class. mAP50 averages AP across classes at IoU 0.50, while mAP50–95 averages over IoU thresholds from 0.50 to 0.95 in steps of 0.05. The latter is stricter because it increasingly rewards well-localised boxes.

The project also evaluates counting, since a field workflow ultimately needs the number of rocks as well as their boxes. For N images, count mean absolute error is

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^{N} |\hat{y}_i - y_i|, \tag{2.3}$$

where y_i and $\hat{y}_i$ are ground-truth and predicted rock counts. Count MAE is deliberately reported alongside mAP rather than used as a replacement: a detector may produce a similar count while assigning an incorrect burial-status class or poorly localising the object.

Metric interpretation also depends on the operating point. A confidence threshold trades recall against false positives, while non-maximum suppression and the IoU setting affect which overlapping candidate boxes survive. Consequently, one confusion matrix or count error is not an intrinsic property of a model independent of threshold choice. The benchmark fixes these settings so that representations are compared consistently; class-specific confusion results are then used to explain failure modes rather than to replace threshold-integrated AP measures.

2.5 Edge Inference, Numerical Precision and Quantisation

Edge computing moves processing closer to the sensor rather than depending on remote cloud inference. In agriculture this can reduce communication latency and dependence on field connectivity. Kalyani and Collier identify lower latency and local processing as important motivations for cloud/fog/edge combinations in smart agriculture [8]. More recent reviews emphasise both the opportunity for local intelligent machinery and the need for stronger measurement of accuracy, latency, memory and resource trade-offs [20]. Raspberry Pi systems are already used across multiple precision-agriculture applications, supporting their role as a low-cost experimentation platform while also leaving scope for more robust deployment research [7].

Model representation strongly influences edge behaviour. FP32 provides a conventional floating-point baseline. FP16 reduces numerical storage and bandwidth, but its speed benefit depends on whether the target runtime and processor can exploit half precision efficiently. INT8 post-training quantisation maps real values to a restricted integer range. A common affine form is

$$q = \text{round}\left(\frac{r}{S}\right) + Z, \tag{2.4}$$

where r is the real value, q the quantised integer, S a scale and Z a zero point. Efficient integer-only inference can substantially reduce computation and memory traffic, but quantisation introduces approximation error [6]. Representative calibration data are therefore important when ranges are estimated after training.

Post-training quantisation is attractive because it avoids retraining the detector, but the calibration set becomes part of the deployment methodology. If representative activation ranges are poorly estimated, clipping or coarse scaling can disproportionately affect difficult examples. The project therefore drew INT8 calibration images only from original training data and balanced the calibration object counts across the two classes. This does not guarantee lossless quantisation, but it prevents the more serious methodological error of calibrating directly on the final test images.

FP16 has a different trade-off. It reduces storage and numerical bandwidth without constraining values to an eight-bit integer grid, so its accuracy is often closer to FP32. However, theoretical savings translate into latency only when the hardware and runtime exploit half-precision operations effectively. The later NCNN result—roughly half the model storage but only a small latency improvement over NCNN FP32—illustrates why numerical format and realised device speed must be measured separately.

2.6 Deployment Runtimes and Inference Benchmarking

The six representations in this study span four software ecosystems. PyTorch FP32 is the original reference representation. ONNX is a portable graph format executed here with ONNX Runtime [13]. TFLite/LiteRT provides deployment-oriented FP32 and INT8 execution [5]. NCNN is a lightweight inference framework designed for mobile and embedded CPU environments and supports both FP32 and FP16 representations [21].

Benchmarking must control the workload if runtime effects are to be interpreted fairly. MLPerf Inference formalises the broader principle that inference comparisons require clear scenarios, metrics and reproducible conditions [17]. This dissertation does not implement the MLPerf suite, but adopts the same discipline: the completed representations process the same 90 images at the same 640×640 input resolution and batch size one, with the same detection threshold settings. Latency, throughput, CPU utilisation, process memory, temperature, detection quality and count error are considered jointly.

Latency and FPS alone are insufficient for a constrained device. A backend can be fast while using high CPU, accumulating heat or failing under repeated inference. Likewise, a highly accurate representation may be too slow for a practical on-the-go workflow. The central analytical idea of this dissertation is therefore an accuracy–efficiency–robustness trade-off rather than a single leaderboard metric.

A controlled benchmark also needs a clear timing boundary. End-to-end camera latency, image preprocessing, model execution and post-processing can all be measured, but combining different boundaries across runtimes would make comparison misleading. The present experiment standardises input geometry in advance and concentrates on the implemented inference path, while separately acknowledging that a future live-camera system will introduce additional acquisition and pipeline costs. Warm-up inference is used because the first call can include graph initialisation, memory allocation or cache effects that are not representative of steady execution.

Mean latency describes the average timed cost, while median FPS provides an intuitive measure of typical throughput. Resource measures add complementary information: process RSS captures memory resident for the running process; CPU utilisation indicates how aggressively the backend uses the processor; and temperature provides a proxy for accumulated thermal load. None is sufficient on its own, which is why the later selection considers the joint evidence.

2.7 Research Gap

The literature establishes three connected points. First, agricultural object detection is useful but sensitive to field variability [1, 2]. Second, local edge processing is attractive where latency and connectivity matter, yet deployment studies need multidimensional resource evaluation [8, 20]. Third, Raspberry Pi platforms are sufficiently capable to support precision-agriculture applications but remain constrained systems [7, 16].

The gap addressed here is consequently not whether YOLO can detect agricultural objects in principle. It is the lack of evidence that a detector selected in a development environment will retain an appropriate combination of accuracy, latency, memory use and operational behaviour when moved into alternative runtime representations on low-cost hardware. Holding the detector, images and target device constant while changing the representation allows the project to isolate this practical deployment decision.

This position also distinguishes the dissertation from studies that report only compressed model size or desktop inference speed. A format that is theoretically portable may still fail because of software compatibility, memory pressure or device-level instability, while a smaller numeric representation may

provide little acceleration if the target CPU cannot exploit it. Conversely, a runtime can provide large speed gains without changing the learned weights materially. Measuring all formats on the Raspberry Pi is therefore necessary to convert general expectations about quantisation and lightweight inference into device-specific evidence.

The review consequently motivates two linked evaluations. The first asks whether the selected detector generalises from validation to unseen project images and whether both burial classes are recognised comparably. The second asks how much of that detection behaviour survives deployment and what speed/resource consequences accompany each representation. Chapters 3 and 4 preserve this separation so that an architecture weakness is not incorrectly blamed on a runtime, and a runtime failure is not incorrectly interpreted as evidence that the detector architecture itself is invalid.

A final background issue is the distinction between model size, process memory and computational cost. A smaller model file reduces storage and transfer requirements, but resident memory also contains runtime libraries and working buffers; similarly, fewer bytes per parameter do not imply a proportional reduction in latency. FLOP-style complexity estimates are useful for architecture comparison, yet they do not capture operator implementation, thread scheduling or memory access on a specific CPU. For that reason, the dissertation treats file/model representation as one factor and relies on measured RSS, CPU utilisation and latency for the deployment decision.

Reproducibility also requires negative outcomes to remain visible. If a runtime exports successfully but resets the device during a longer workload, excluding it without explanation would bias the comparison toward successful formats. Equally, filling the missing row with estimates from a few images would make the table appear complete at the expense of validity. The methodological position adopted here is therefore conservative: only completed common-workload runs receive aggregate metrics, and incomplete or failed runs are reported qualitatively with their evidential limits. This principle is particularly important for embedded systems, where software compatibility and operational stability are part of deployment performance rather than peripheral implementation details.

Chapter 3

Design and Implementation

3.1 Implementation Overview

The implementation consisted of two connected stages. The first established a two-class detector for identifying rocks in agricultural RGB images and distinguishing completely exposed from half-buried rocks. The second transferred the selected detector to a Raspberry Pi 5 and evaluated alternative representations under a controlled stored-image workload.

3.2 Dataset Preparation and Detector Selection

The original dataset contained 448 images and 706 annotated rocks. Before augmentation it was partitioned into training, validation and test sets as shown in Table 3.1. The test set contained 90 images and 135 labelled rocks and was not used to select the detector architecture.

Table 3.1: Dataset split and class composition before augmentation.

Split	Images	Completely exposed	Half buried
Training	313	259	250
Validation	45	34	28
Test	90	69	66
Total	448	362	344

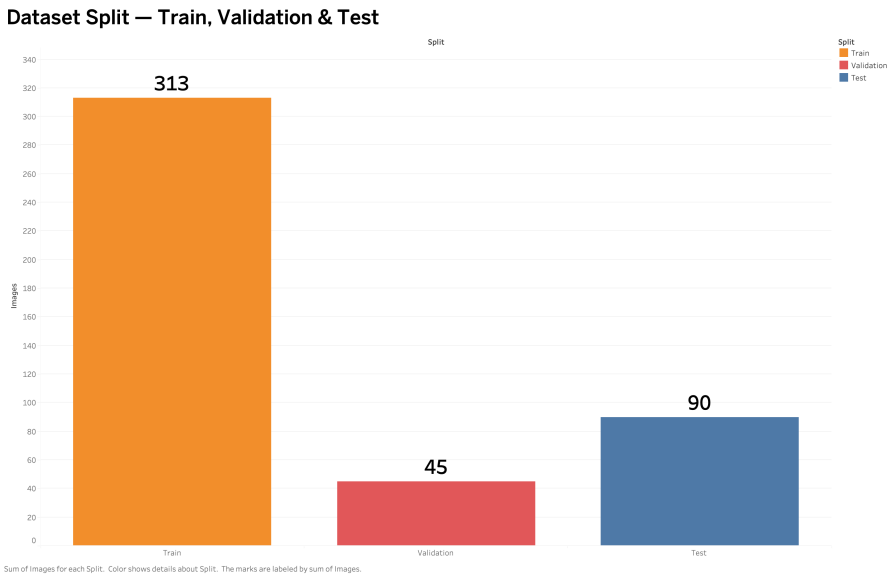


Figure 3.1: Distribution of the 448 original images across the training, validation and test partitions. The fixed pre-augmentation split separates detector fitting from architecture selection and the later 90-image evaluation workload.

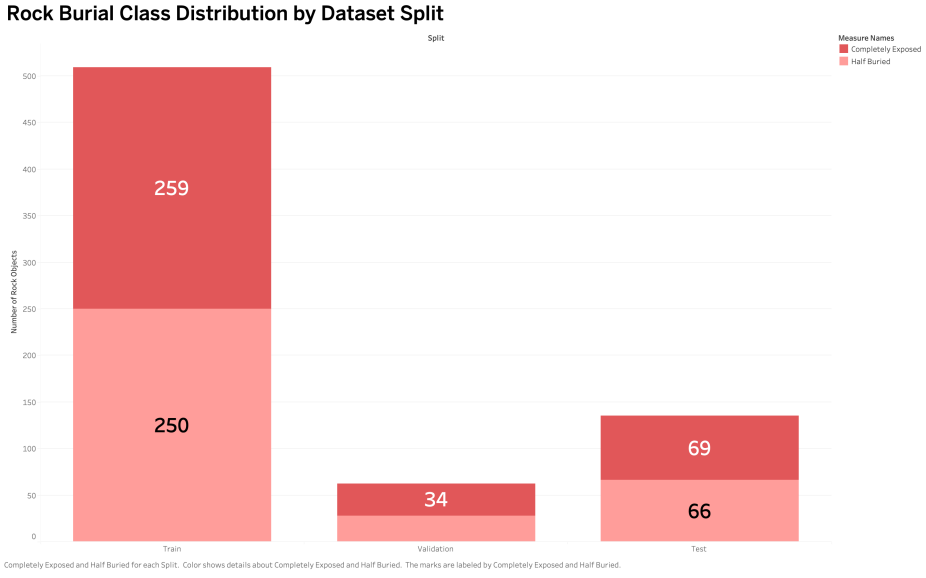


Figure 3.2: Class-level annotation counts for completely exposed and half-buried rocks in each dataset split. The closely balanced overall totals allow later performance differences to be interpreted primarily as differences in visual difficulty rather than severe label imbalance.

Only the training set was expanded offline. Each of the 313 training images received six photometric variants, producing 1,878 augmented images in addition to the originals. The resulting 2,191-image training set contained 3,563 annotation instances across augmented copies. The augmentation strategy targeted plausible appearance variation while avoiding geometric changes that could undermine the existing boxes.

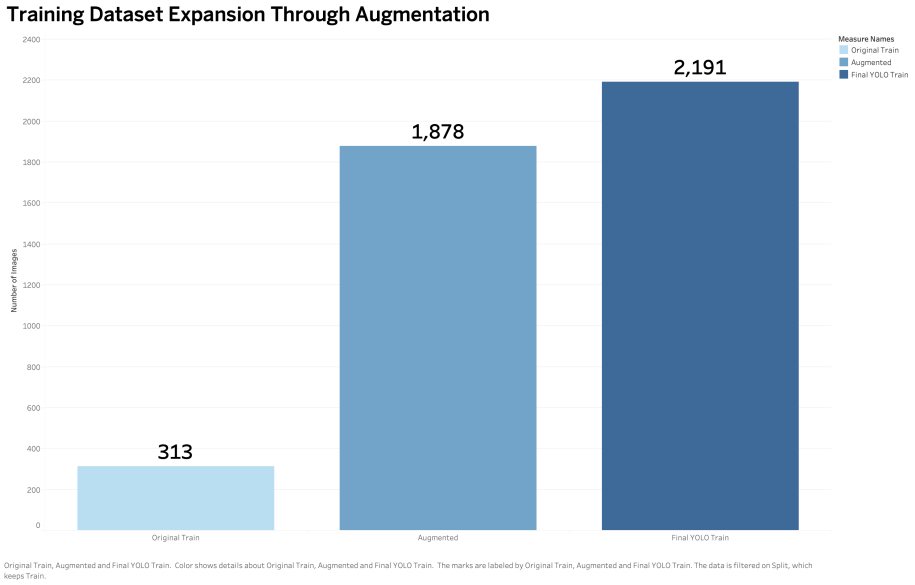


Figure 3.3: Expansion of the training partition from 313 original images to 2,191 images after six photometric variants were generated per source image. Only training data were augmented; validation and test images remained unchanged.

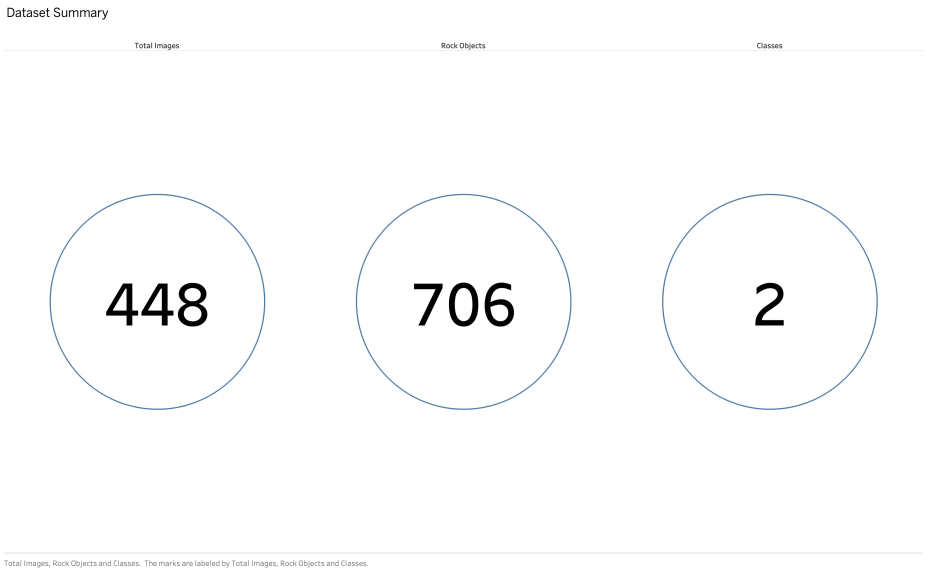


Figure 3.4: Dataset-scale summary showing 448 original images, 706 annotated rock objects and two burial-status classes. These figures define the limited-data setting that motivates transfer learning, controlled augmentation and careful held-out evaluation.

YOLOv8n, YOLO11n and YOLO11s were trained under a common main configuration: a maximum of 20 epochs, image size 896, batch size 32, seed 42, automatic mixed precision, cosine learning-rate scheduling and Ultralytics automatic optimiser selection, which selected AdamW. Mosaic was disabled; horizontal flip probability was 0.5, translation 0.04 and scale 0.1. Final comparative validation inference used 1024-pixel images.

The common training settings were intended to make the architecture comparison interpretable. The

candidate models were not given independently tuned hyperparameters that could favour one model. Training curves and validation outputs were retained for each candidate, and the final selection was made only after the common validation inference. This design is less exhaustive than a full hyperparameter search, but it provides a controlled comparison appropriate to the project scope.

Table 3.2 reports the validation comparison. YOLO11s achieved the best F1, recall and mAP50, but YOLOv8n achieved the highest mAP50–95. Because mAP50–95 was defined in advance as the primary architecture-selection criterion, YOLOv8n was selected. This avoids a post-hoc claim that the chosen model was "best" on every measure.

Table 3.2: Validation comparison used for detector selection.

Model	Precision	Recall	F1	mAP50	mAP50–95
YOLOv8n	0.681	0.759	0.718	0.759	**0.4288**
YOLO11n	0.743	0.719	0.731	0.729	0.4002
YOLO11s	0.764	**0.801**	**0.782**	**0.783**	0.4107

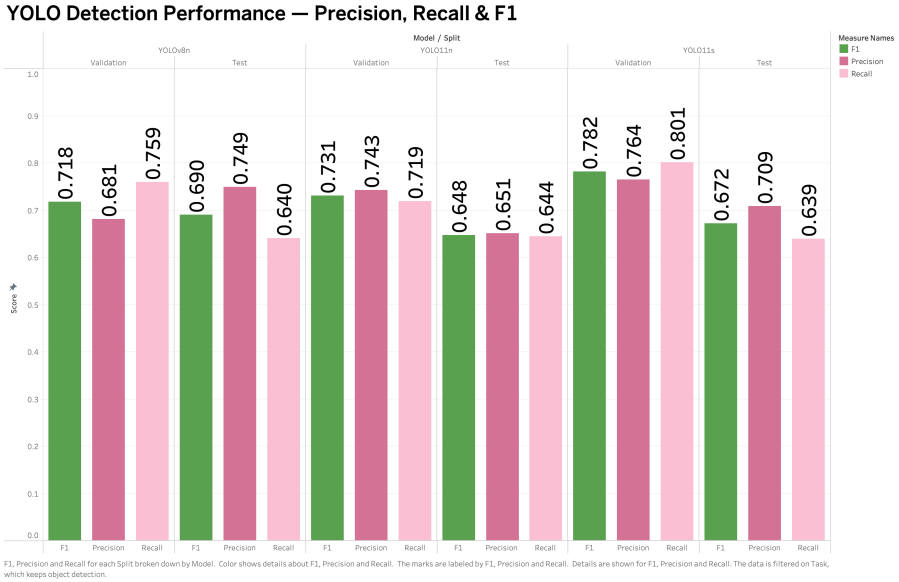


Figure 3.5: Validation and test precision, recall and F1 for YOLOv8n, YOLO11n and YOLO11s. YOLO11s leads several operating-point metrics, showing why the detector choice cannot be justified from a single score in isolation.

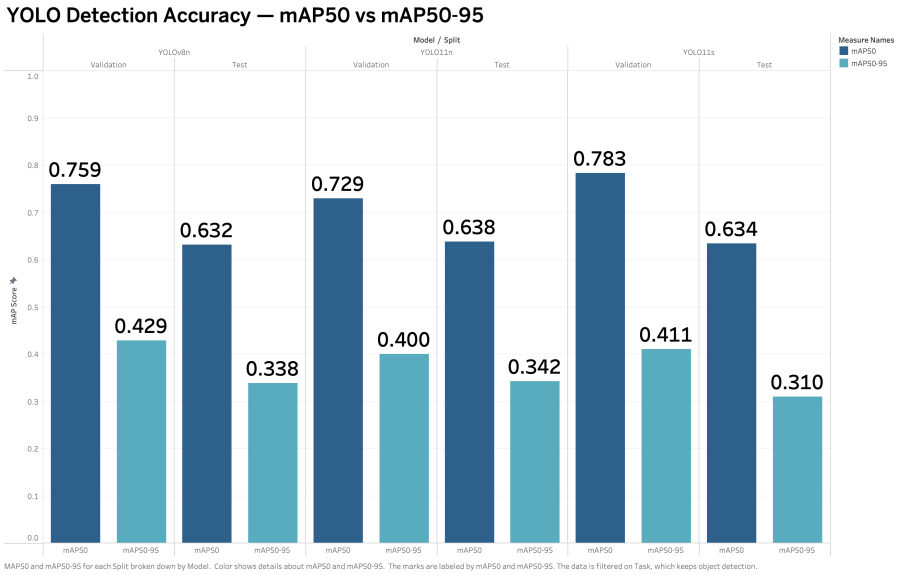
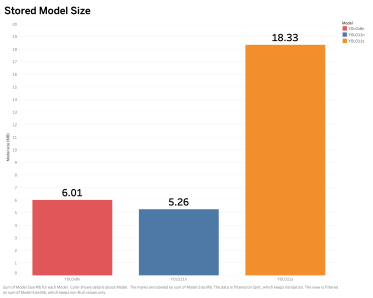
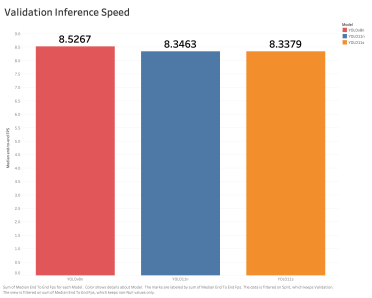


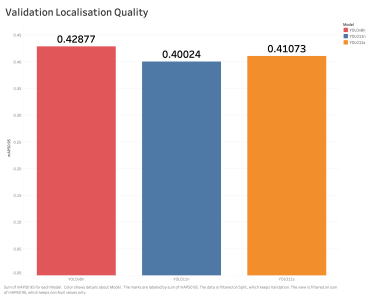
Figure 3.6: mAP50 and mAP50–95 for the three candidate YOLO detectors on validation and test data. The stricter mAP50–95 measure evaluates localisation across multiple IoU thresholds and was the predefined architecture-selection criterion.



(a) Stored model size. YOLO11n has the smallest stored footprint at 5.21 MB, YOLOv8n remains compact at 6.00 MB, and YOLO11s is substantially larger at 18.35 MB.



(b) Validation inference speed. Development-side throughput is nearly identical across the candidates, with YOLOv8n marginally highest at 8.53 FPS.



(c) Validation localisation quality. YOLOv8n records the strongest validation mAP50–95 at 0.4288, the predefined detector-selection criterion, ahead of YOLO11s and YOLO11n.

Figure 3.7: YOLO candidate efficiency and validation quality.

Evaluation of the selected YOLOv8n on the untouched architecture-selection test set produced mAP50–95 of approximately 0.3384, compared with 0.4288 on validation. The absolute reduction of 0.0904 (about 21% relative) indicated a meaningful generalisation gap and motivated caution when interpreting all later deployment results.

The ordering of these steps is important. Architecture choice was fixed from validation before the test value was examined, so the 90-image partition retained its intended role for estimating the selected architecture at that stage. Once the same 90 images were later reused to compare edge representations and choose a runtime, however, they ceased to be a completely untouched final deployment test. The dissertation therefore distinguishes the validity of the original architecture-selection use from the later limitation introduced by runtime selection.

The model-development records also retained class-level outputs, confusion information and saved weights rather than only a single best score. This made it possible to identify the half-buried weakness before deployment. Without that evidence, a lower class performance observed after INT8 conversion could easily have been misattributed to quantisation rather than recognised as a pre-existing detector limitation.

A separate CLIP ViT-B/32 branch investigated whether pretrained vision–language embeddings could distinguish burial status from rock crops [14]. This work remained a supporting model-development branch. No verified Raspberry Pi CLIP benchmark was completed, and the deployment experiments below concern YOLOv8n only.

The research branch used CLIP ViT-B/32 in two modes: zero-shot classification using textual class descriptions, and a lightweight linear probe trained on frozen image embeddings. A confidence- based rule was explored for disagreements between the YOLO class and CLIP classification, with a configured CLIP override threshold of 0.80. Conceptually this created a pipeline of *field image* → *YOLO detection* → *rock crop* → *CLIP verification*. The branch was intentionally excluded from the final Pi comparison because running CLIP on every detected crop would add a second model, additional memory demand and extra latency, and no complete device benchmark existed to quantify that cost.

3.3 YOLO Detection and Output Processing

The selected YOLOv8n detector is the primary executable model in the final system. For each input image it produces a collection of bounding boxes, confidence values and class identifiers in one forward pass. During development, the common training and validation configuration was used to compare candidate architectures. During edge evaluation, the selected weights received the same fixed 640×640 letterboxed pixels in every runtime so that the comparison did not confound detector behaviour with different image geometry.

Post-processing used the common confidence threshold of 0.25 and IoU setting of 0.70. Candidate boxes were converted into the common coordinate system, associated with either the completely exposed or half-buried class, and retained in structured per-image outputs. These predictions drove the annotated images, class-specific counts and the precision, recall, F1 and mAP calculations. Treating the output as structured detection evidence, rather than only a rendered image, made it possible to trace aggregate results back to individual images and class-specific errors.

The two-class detection formulation also avoids requiring a second classifier for the deployed benchmark: localisation and burial status are produced by the same compact network. This reduces pipeline complexity and keeps runtime comparison focused on one trained model. The trade-off is that weak visual evidence for a half-buried rock can affect both localisation and class assignment, which is why the report gives the detector-selection evidence and class-specific behaviour greater weight than an FPS-only deployment comparison.

Holding the YOLOv8n weights, input pixels, thresholds and output interpretation constant across PyTorch, TFLite and NCNN is the principal experimental control. Observed changes can therefore be analysed as representation and runtime effects while retaining the same underlying detection task.

3.4 Edge-Deployment Design and Test Workload

The selected YOLOv8n model was prepared in six representations: PyTorch FP32, ONNX FP32, TFLite FP32, TFLite INT8, NCNN FP32 and NCNN FP16. These formats were chosen to span the original framework, an exchange/runtime path, an integer-quantised deployment path and a lightweight embedded inference framework.

All six variants originated from the same selected YOLOv8n weights. They were not separately retrained after export. This is a key control in the experiment: when a converted representation changes accuracy or latency, the difference is primarily associated with conversion, numerical precision and backend execution rather than a different optimisation history. Export success was also separated from deployment success. A file existing on disk or loading once was not sufficient to classify a representation as benchmark-complete.

For the edge comparison, the same 90 test images were converted into fixed 640×640 letterboxed images. If the original dimensions are $W \times H$, the aspect-ratio-preserving scale is

$$s = \min\left(\frac{640}{W}, \frac{640}{H}\right), \tag{3.1}$$

with symmetric padding added using fill value 114. This produces a common input shape while avoiding aspect-ratio distortion. The 90 images contain 135 labelled rocks: 69 completely exposed and 66 half buried.

Bounding-box centres and dimensions were transformed using the same scale and padding offsets as the image. A manifest retained original dimensions, deployment dimensions, scaling and padding information so that the fixed images could be traced back to their sources. Pre-generating the letterboxed workload reduced framework-specific preprocessing as a confounding variable: each backend received the same pixels at the same geometry instead of relying on slightly different runtime resize implementations.

A ten-image smoke-test subset was copied from these same 90 images to verify model loading, input/output handling and basic post-processing before longer runs. It is therefore a functional subset, not an independent evaluation dataset.

The smoke-test procedure checked model loading, image acceptance, inference, class identifier mapping, bounding-box generation, class-specific counting, annotated-image output and structured JSON output. In the final functional check, the optimised formats returned the expected single completely exposed rock on the representative image. Passing this test established interface compatibility only; it did not contribute additional accuracy samples beyond the 90-image set.

The INT8 conversion used 200 *original training images* as representative calibration data, not validation or test images. The selected calibration set contained 304 labelled objects, balanced at 152 completely exposed and 152 half buried. This separation prevents direct leakage of the final test images into calibration while ensuring that both classes influence activation range estimation.

The calibration set contained 304 labelled objects in total. Equal object counts by class do not mean the calibration data are statistically identical to the test distribution, but the design ensures that one burial class does not dominate simply because it contributes more representative objects. After conversion, the calibration images themselves were not required for device inference; provenance was retained through the calibration manifest rather than by treating the calibration subset as a second evaluation set.

3.5 Raspberry Pi Environment and Tooling

The target device was a Raspberry Pi 5 Model B with a 64-bit ARM environment and approximately 8 GB RAM [16]. It ran Debian GNU/Linux 13 and Python 3.13.5. Key package versions were Ultralytics 8.4.115, ONNX Runtime 1.28.0, OpenCV 5.0.0, NumPy 2.5.1, Pandas 3.0.5 and psutil 7.2.2. LiteRT inference used the `ai_edge_litert.interpreter` interface. NCNN was used for the FP32 and FP16 variants.

Power was supplied by the official Raspberry Pi 27 W USB-C supply rated at 5.1 V/5 A. No Active Cooler was available in the final configuration; Raspberry Pi documentation treats cooling as a material consideration for sustained high-load operation [15]. No live camera was used. A MacBook served only for SSH/SCP control and file transfer; measured inference ran on the Pi CPU.

Recording exact versions is important because inference behaviour is not determined by the model file alone. Runtime releases, Python bindings, numerical libraries and operating-system changes can alter performance or compatibility. The environment table therefore forms part of the reproducibility evidence rather than merely documenting convenience software.

Deployment scripts performed model-specific loading, image reading, inference, post-processing, resource sampling and CSV/JSON output. Output manifests and saved predictions were used to verify that each completed run contained the intended 90 images rather than silently omitting failures.

The tooling was separated by purpose. Environment verification recorded software versions before benchmarking; single-image inference provided a low-cost functional test; the benchmark script produced

Table 3.3: Key Raspberry Pi benchmark environment.

Component	Recorded configuration
Device	Raspberry Pi 5, aarch64, approximately 8 GB RAM
Operating system	Debian GNU/Linux 13
Python / Ultralytics	Python 3.13.5 / Ultralytics 8.4.115
ONNX Runtime	1.28.0
OpenCV / NumPy / Pandas	5.0.0 / 2.5.1 / 3.0.5
Resource monitoring	psutil 7.2.2
LiteRT interface	`ai_edge_litert.interpreter`
Power	Official Raspberry Pi 27 W, 5.1 V/5 A
Cooling	No Active Cooler
Input source	Stored images; no camera

per-image timing/resource records and aggregate outputs; and a dedicated stability script performed repeated continuous inference. SSH and SCP scripts supported transfer between the MacBook and Pi without moving the actual inference workload off-device. This separation reduced manual intervention and made it easier to distinguish an environment problem, a model-interface problem and a sustained-runtime problem.

For timing, model loading was kept outside the principal per-image inference measurement where possible, and warm-up calls were used before the timed pass. CPU and process RSS were sampled as system-level indicators rather than interpreted as exact energy measurements. Temperature was recorded from the Raspberry Pi monitoring interface. These measurements therefore characterise the implemented software stack on the Pi, not an abstract hardware-only cost of the neural network.

3.6 Benchmark and Evaluation Protocol

The final executed benchmark used batch size one, 640×640 inputs, confidence threshold 0.25 and IoU setting 0.70. Each representation received three warm-up inferences followed by one timed pass across the 90-image workload. Table 3.4 distinguishes this from an earlier planning document that proposed ten warm-ups, three timed passes and a 30-minute stability test. Those planned settings were not executed and are not used as if they were observed evidence.

Table 3.4: Planned protocol versus the protocol actually executed.

Element	Earlier plan	Executed benchmark
Input size	640×640	640×640
Batch size	1	1
Warm-up	10 inferences	3 inferences
Timed evaluation	3 passes	1 pass
Images per completed format	90 per pass	90
Stability target	30 minutes	Separate short tests only
Completed formats	Planned six	Five complete; ONNX aggregate N/A

For each completed representation, the benchmark recorded inference latency, derived throughput, CPU utilisation, process resident memory, temperature and predictions. Detection outputs were then evaluated against the common labels to obtain precision, recall, F1, mAP50, mAP50–95 and count MAE. Median FPS is reported consistently in the final comparison because it is less sensitive to isolated timing spikes than an arithmetic mean of reciprocal latencies.

Latency was recorded per image after warm-up, allowing both aggregate means and the distribution of individual timings to be inspected. Throughput was derived from the observed inference rate rather than from a theoretical operation count. Process memory was tracked as RSS, which captures the resident memory of the running process but not every system allocation attributable indirectly to the workload. CPU percentage is likewise a software/runtime measure: a backend can achieve lower latency by using more of the available cores, so high CPU utilisation is interpreted together with speed rather

than automatically penalised.

The prediction record retained class labels, confidences and bounding boxes for downstream accuracy validation. Rock count was obtained from the retained detections per image, making count MAE directly traceable to the same predictions used for mAP. Temperature and throttle indicators were captured during benchmark/stability runs so that a high FPS result could be checked for signs of unsustainable operating conditions.

The accuracy evaluation used the predictions saved by each completed representation rather than mixing benchmark timings from one run with detections from a different workload. This allowed the same labels to support both detection metrics and per-image rock-count error. It also made failure visible: a representation that did not produce the complete prediction set could not be assigned an aggregate row by extrapolating from a small subset.

3.7 Reliability, Diagnostics and Reproducibility

Five representations completed the 90-image benchmark: PyTorch FP32, TFLite FP32, TFLite INT8, NCNN FP32 and NCNN FP16. ONNX FP32 exported correctly and processed isolated images, but repeated sustained attempts ended in abrupt Raspberry Pi reboots before a valid aggregate run was produced. The root cause was not established, so no ONNX aggregate latency, accuracy or resource values are estimated. The defensible conclusion is only that ONNX FP32 was functionally viable for isolated inference but did not demonstrate stable sustained benchmarking in the tested configuration.

Diagnostic logs were reviewed for evidence that could distinguish an application exception from a system reset. The available records established the temporal association between sustained ONNX attempts and abrupt reboot, but did not uniquely identify thermal throttling, undervoltage, out-of-memory termination or an ONNX Runtime defect. This is why the final benchmark table marks ONNX as N/A instead of reporting values from a partial run. Preserving an incomplete row is less visually satisfying than a six-way comparison, but avoids false precision.

Operational testing also exposed the difference between completing a short fixed workload and remaining stable continuously. NCNN FP32 completed the 90-image benchmark but a later continuous run hard-reset the Pi and did not produce a valid completed continuous CSV. By contrast, TFLite INT8 completed a genuine continuous experiment lasting approximately two minutes, producing about 2,533 inference rows at roughly 22.7 FPS. Temperature increased from the mid-40s to about 76–77 °C, with a raw recorded maximum near 77.1 °C, and no throttling or reboot was recorded during that run.

This is deliberately described as short-duration evidence. No valid 10-minute or 30-minute final stability run exists. A file whose name suggested a 30-minute test was found to duplicate the shorter dataset and is not treated as independent evidence. This distinction between planned, completed and invalid evidence is part of the reproducibility approach rather than a limitation to be hidden.

Reproducibility was supported by retaining the deployment folder structure, environment records, input manifests, prediction JSON files, annotated outputs and benchmark CSVs. The evidence was also checked against duplicate backups so that copied result folders were not counted as separate experiments. Where a long-run file conflicted with its contents, the contents and row counts took precedence over the filename. This evidence-first rule is why the dissertation reports one genuine approximately two-minute INT8 stability run and no completed 10- or 30-minute final run.

The same principle was applied to backup folders. Copies of a benchmark were treated as backups, not additional experimental repetitions. Annotated-image and prediction-JSON counts were used to confirm the expected 90-image completion for formats such as NCNN FP16. This prevents the number of files in an archive from being mistaken for the number of independent trials.

3.8 Chapter Summary

The implementation created a controlled path from an empirically selected YOLOv8n detector to five fully benchmarked Raspberry Pi representations plus an ONNX format that remained incomplete under sustained testing. The next chapter evaluates what those measurements mean, with particular attention to the pre-existing half-buried weakness and the trade-off between PyTorch accuracy, INT8 speed and NCNN FP16 accuracy retention.

Chapter 4

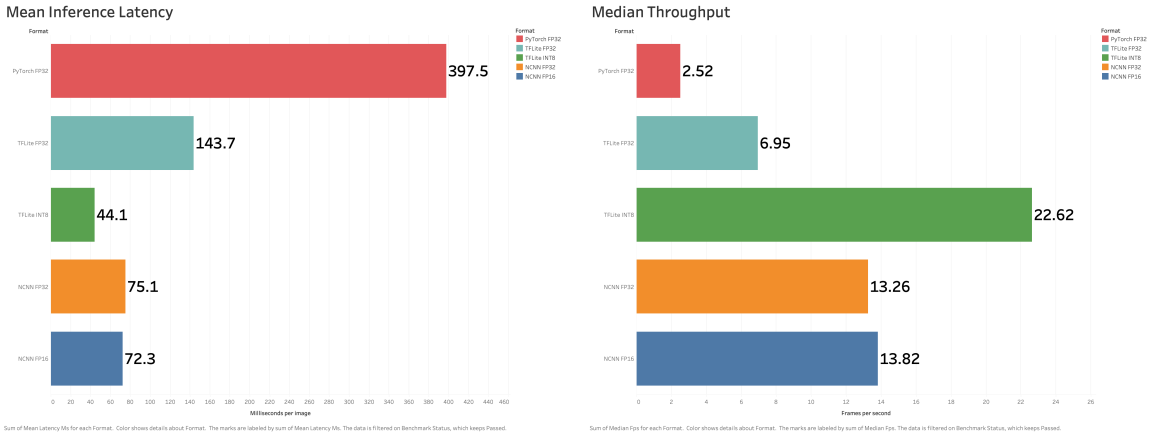
Critical Evaluation

4.1 Evaluation Overview

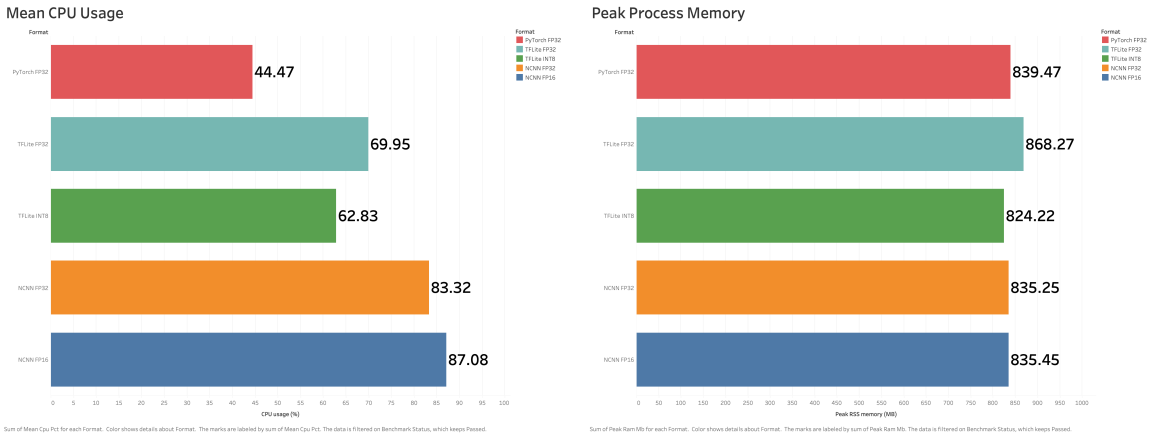
The central evaluation result is that deployment quality cannot be represented by a single metric. The representation with the strongest measured detection quality was the slowest, while the fastest representation introduced a measurable accuracy penalty. Resource utilisation and operational behaviour also differed between backends. Table 4.1 therefore presents the core cross-format evidence in one place.

Table 4.1: Canonical Raspberry Pi benchmark results. FPS denotes median throughput.

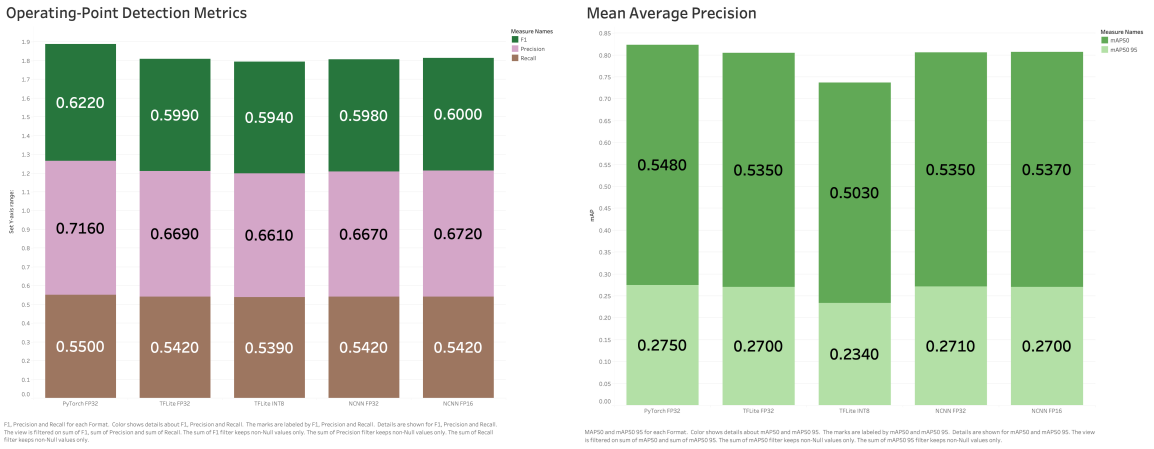
Representation	Latency ms	FPS	CPU %	Peak RAM MB	P	R	F1	mAP50	mAP50–95
PyTorch FP32	397.48	2.52	44.47	839.47	0.716	0.550	0.622	0.548	0.275
TFLite FP32	143.70	6.95	69.95	868.27	0.669	0.542	0.599	0.535	0.270
TFLite INT8	44.13	22.62	62.83	824.22	0.661	0.539	0.594	0.503	0.234
NCNN FP32	75.09	13.26	83.32	835.25	0.667	0.542	0.598	0.535	0.271
NCNN FP16	72.31	13.82	87.08	835.45	0.672	0.542	0.600	0.537	0.270
ONNX FP32	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A



(a) Runtime performance: mean inference latency and median throughput for the five completed representations.



(b) Resource demand: mean CPU usage and peak process memory measured during the same 90-image benchmark.



(c) Detection quality: precision, recall, F1, mAP50 and mAP50–95 retained after deployment conversion.

Part 1 – Efficiency and resources. Images (a) and (b) compare the runtime trade-off across latency, throughput, CPU usage and peak process memory; TFLite INT8 provides the strongest speed and lowest memory result, while the NCNN formats require the greatest CPU share.

Part 2 – Detection quality. Image (c) separates operating-point precision, recall and F1 from the stricter mAP measures. PyTorch FP32 retains the strongest aggregate accuracy, whereas TFLite INT8 exchanges some localisation quality for substantially faster inference. ONNX FP32 is omitted because no valid aggregate benchmark was completed.

Figure 4.1: Three-image graphical summary of Table 4.1.

Count MAE was 0.600 for PyTorch FP32, TFLite FP32 and NCNN FP32, 0.611 for NCNN FP16 and 0.633 for TFLite INT8. The close values indicate that quantisation did not create a catastrophic counting failure, but the modestly higher INT8 error is consistent with its lower detection metrics.

4.2 Detector Selection and Class-Specific Behaviour

The detector-selection result should be interpreted according to the stated criterion. YOLO11s had the highest validation mAP50 (0.783), recall (0.801) and F1 (0.782), whereas YOLOv8n had the highest validation mAP50–95 (0.4288). Selecting YOLOv8n was therefore internally valid because mAP50–95 had been defined as the primary criterion; it would be incorrect to claim that YOLOv8n was best across every metric.

The untouched test result also shows why validation leadership should not be mistaken for final generalisation. YOLOv8n mAP50–95 decreased from 0.4288 on validation to approximately 0.3384 on the untouched test set. A 0.0904 absolute drop, around 21% relative, suggests that the modest dataset and heterogeneous field appearance leave material uncertainty beyond the development partition. This is consistent with literature that identifies generalisation under changing agricultural conditions as a persistent challenge [1, 2].

Several mechanisms could contribute to this gap: the small validation partition may provide a noisy estimate, the held-out images may contain harder scenes, or the selected training recipe may not cover the full variation in soil, illumination and occlusion. The experiment does not isolate which explanation dominates. The appropriate conclusion is therefore that the validation result was optimistic relative to the held-out test, not that a specific data defect or optimisation choice was proven to cause the reduction.

The largest semantic weakness was class-specific. At the evaluated source-detector operating point, 64 of 69 completely exposed rocks were correctly represented in the relevant confusion result (about 92.8%), compared with only 23 of 66 half-buried rocks (about 34.8%). These are operating-point correct rates from the confusion analysis, not AP values. Their importance is that the burial problem existed before Raspberry Pi conversion. INT8 and FP16 can alter confidence or localisation, but they cannot plausibly be treated as the sole cause of a weakness already visible in the source detector.

The result has a direct design implication: future effort spent only on runtime optimisation may improve FPS while leaving the main application-level recognition weakness substantially unchanged. More representative half-buried examples, stronger hard-negative coverage and perhaps sensing modalities less sensitive to soil/stone appearance are likely to produce greater field value than further micro-optimisation of an already fast INT8 runtime.

This asymmetry also affects how aggregate mAP should be interpreted operationally. An average over two classes can hide the fact that one class is close to the intended behaviour while the other is frequently missed. In a rock-removal workflow, missing a half-buried stone may be more costly than a modest localisation error on a fully exposed stone. Future evaluation should therefore report per-class AP/recall and field-relevant miss rates alongside aggregate metrics, and should consider whether class-specific confidence thresholds are justified.

4.3 Accuracy–Efficiency Trade-offs Across Edge Formats

Relative to the 397.48 ms PyTorch baseline, TFLite FP32 reduced mean latency by approximately 2.77×, NCNN FP32 by 5.29×, NCNN FP16 by 5.50×, and TFLite INT8 by 9.01×.

PyTorch FP32 achieved the highest edge-benchmark mAP50 at 0.548 and mAP50–95 at 0.275, but 2.52 median FPS is restrictive for an on-the-go vision component. TFLite FP32 preserved most of that accuracy (mAP50 0.535) while raising throughput to 6.95 FPS. NCNN FP32 produced almost the same accuracy (mAP50 0.535; mAP50–95 0.271) at 13.26 FPS, demonstrating that runtime choice alone can yield a large speed improvement without material loss in the measured accuracy.

NCNN FP16 produced mAP50 0.537 and mAP50–95 0.270 at 13.82 FPS. Its model storage was roughly 49% smaller than NCNN FP32, yet latency improved by only about 3.8%. This is an important counterexample to the assumption that halving numerical width necessarily produces a comparable CPU speedup. On this Raspberry Pi/runtime combination, storage reduction was much larger than execution-time reduction.

The closeness of the three non-quantised deployment results is also notable. TFLite FP32 and NCNN FP32 both achieved mAP50 0.535, while NCNN FP16 slightly increased this to 0.537; all three were within 0.013 mAP50 of the PyTorch baseline. Their mAP50–95 values (0.270, 0.271 and 0.270) were similarly

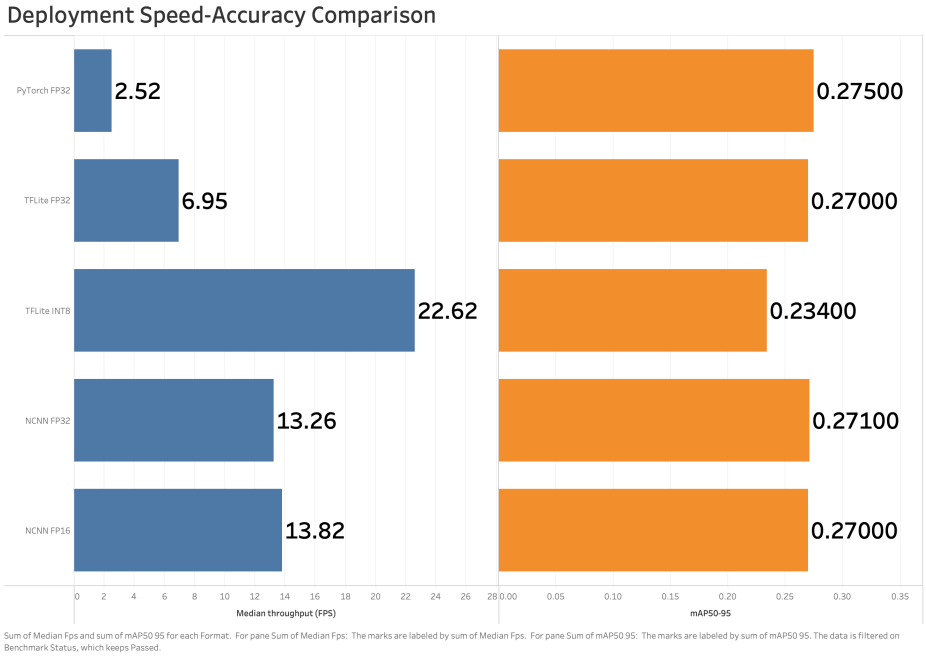


Figure 4.2: Median throughput plotted against mAP50–95 for each deployment format that completed the Raspberry Pi benchmark. Points farther right are faster and points higher on the chart retain stronger localisation quality; ONNX FP32 is omitted because no valid aggregate benchmark was completed.

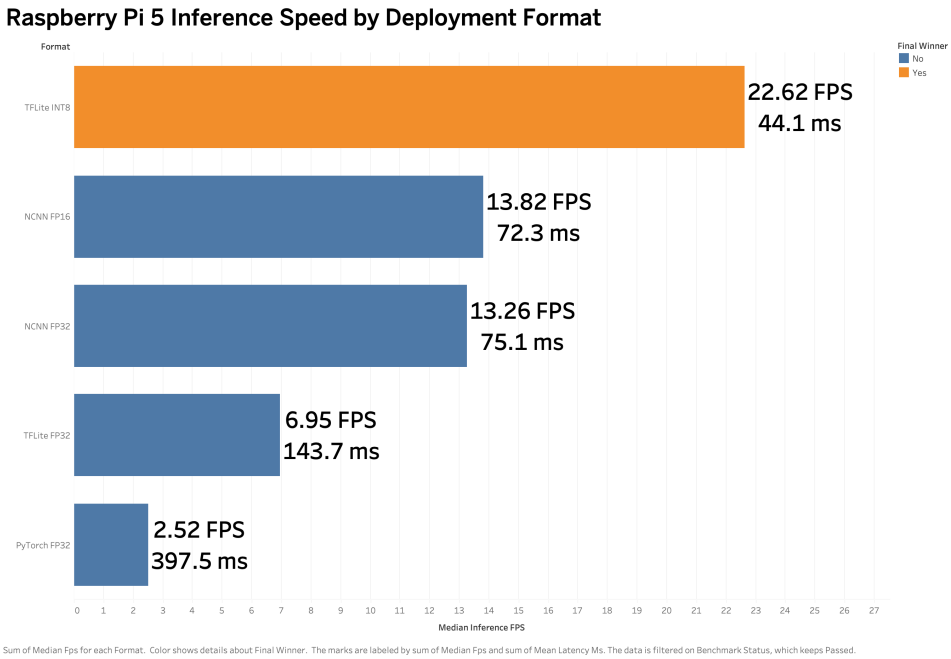


Figure 4.3: Median throughput and corresponding mean inference latency for each completed deployment format on Raspberry Pi 5. TFLite INT8 provides the highest speed at 22.62 FPS and the lowest latency at 44.13 ms.

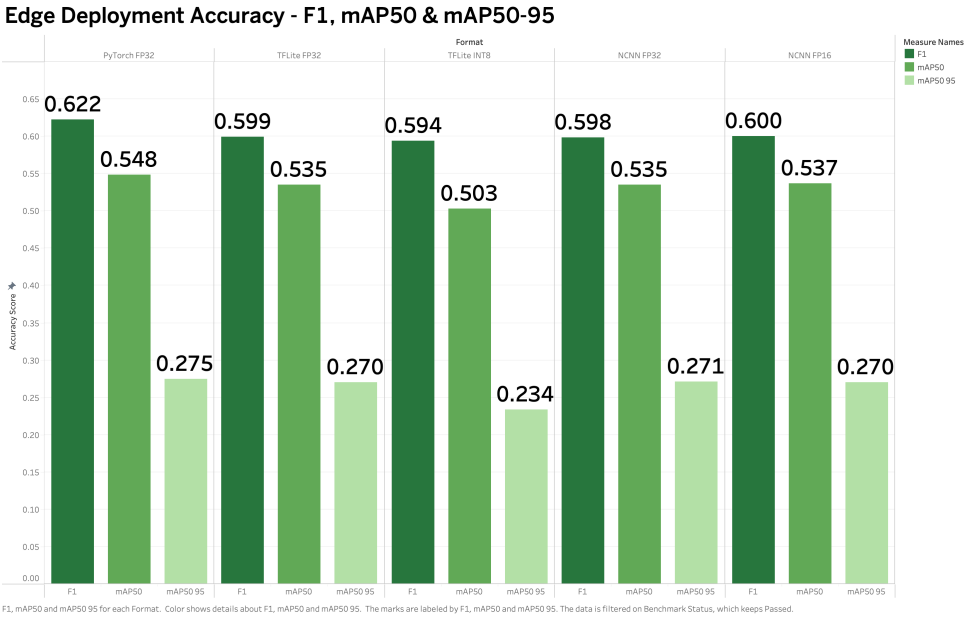


Figure 4.4: F1, mAP50 and mAP50–95 for the five deployment formats that completed the 90-image benchmark. PyTorch retains the strongest aggregate accuracy, while INT8 trades measurable localisation quality for substantially faster inference.

close to PyTorch at 0.275. Within the resolution of this experiment, exporting to these representations preserved most of the detector's aggregate quality. That strengthens the argument that runtime optimisation can produce meaningful speed improvements before accepting the larger quantisation penalty of INT8.

Precision and recall changed only modestly across these three formats. PyTorch precision was 0.716 and recall 0.550; the converted FP32/FP16 formats clustered around 0.667–0.672 precision and 0.542 recall. TFLite INT8 moved to 0.661 precision and 0.539 recall. The direction is consistent with a small degradation rather than a wholesale change in detector behaviour, but the lower mAP50–95 indicates that stricter localisation quality is more sensitive than the headline operating-point recall alone suggests.

TFLite INT8 delivered the largest performance change: 44.13 ms mean latency and 22.62 median FPS, approximately nine times the PyTorch speed. The cost was a decline in mAP50 from 0.548 to 0.503 (0.045 absolute, about 8.2%) and in mAP50–95 from 0.275 to 0.234 (0.041 absolute, about 14.9%). This behaviour is consistent with the general quantisation trade-off described in the literature: integer arithmetic can make inference substantially cheaper, while approximated weights/activations can reduce task accuracy [6].

The practical question is therefore whether the lost accuracy is acceptable for the intended stage of the system. For a proof-of-concept CPU-only edge detector, moving from 2.52 to 22.62 FPS changes the operational character of the application. The corresponding mAP loss is real and must not be minimised, but it is sufficiently bounded that INT8 remains a credible choice when latency is a first-order requirement.

The FP32 runtime comparison is particularly informative because it separates numerical precision from backend implementation. PyTorch FP32, TFLite FP32 and NCNN FP32 all use nominally full precision, yet mean latency changes from 397.48 to 143.70 to 75.09 ms. A large part of the edge speed-up therefore comes from the inference engine and graph/runtime optimisation before reduced precision is introduced. INT8 adds a further major gain on top of this runtime effect. This staged interpretation is more defensible than attributing the entire 9.01× improvement to quantisation alone.

The result also shows why throughput should be compared using the same batch-one scenario. Large batches can increase hardware utilisation on servers but do not represent a camera-like stream in which each frame needs a prompt result. At 22.62 median FPS, the INT8 representation has a nominal per-frame budget compatible with substantially more responsive processing than the 2.52 FPS PyTorch baseline. Whether that is sufficient for a moving planter depends on camera field of view, vehicle speed and required overlap between observations—quantities not measured here—so the study stops short of translating FPS into a guaranteed maximum tractor speed.

Accuracy differences should likewise be treated as estimates on 90 images rather than exact population

values. The reported decimal metrics are useful for comparing completed runs on a common set, but the experiment was not large enough to establish that a difference of a few thousandths between TFLite FP32 and NCNN FP16 would reproduce across independent farms. The broad patterns are more defensible: PyTorch is slowest and most accurate in the aggregate; INT8 is much faster with a clear accuracy penalty; and the NCNN formats retain most FP32 accuracy while providing intermediate throughput.

4.4 Resource, Counting and Thermal Behaviour

Peak process memory varied less dramatically than latency. TFLite INT8 had the lowest measured peak at 824.22 MB. PyTorch, NCNN FP32 and NCNN FP16 were clustered near 835–839 MB, while TFLite FP32 reached 868.27 MB. The absence of a large memory reduction for every compact numeric format reinforces that process RSS includes runtime libraries, buffers and interpreter overhead in addition to model parameters.

Counting behaviour provides an application-facing check on these detection differences. The three FP32 implementations each produced count MAE 0.600, NCNN FP16 0.611 and TFLite INT8 0.633. The small spread suggests broad preservation of total-rock counting, but it does not imply that individual errors are interchangeable. A missed half-buried rock and an additional false-positive exposed rock can partially cancel in image-level count while still representing two detection mistakes. Qualitative review of dense and ambiguous scenes is therefore useful when diagnosing why count error changes, although no unverified example montage is introduced into the main body.

CPU utilisation shows a similarly runtime-specific profile. PyTorch FP32 averaged 44.47%, TFLite FP32 69.95%, TFLite INT8 62.83%, NCNN FP32 83.32% and NCNN FP16 87.08%. A higher CPU percentage is not automatically worse if it accompanies much lower latency; rather, it indicates how aggressively the backend uses available processing capacity. The NCNN formats achieved strong throughput while showing the highest mean CPU utilisation, whereas INT8 achieved the best throughput without the highest CPU figure.

This matters for system integration. A runtime that occupies most CPU capacity may leave less headroom for camera handling, GPS processing, communication or control logic even if its raw inference latency is attractive. Conversely, unused CPU in the PyTorch result does not make it preferable when the achieved throughput is too low. A future end-to-end system should therefore measure total application utilisation after all concurrent components are integrated, rather than assuming the isolated detector profile will remain unchanged.

During the fixed 90-image benchmark, no recorded thermal throttling occurred. Approximate maximum temperatures were 55.10 °C for TFLite INT8, 56.75 °C for NCNN FP32, 59.50 °C for NCNN FP16, 61.70 °C for PyTorch and 66.10 °C for TFLite FP32. These short-run maxima should not be interpreted as equilibrium temperatures.

The continuous test rose from the mid-40s to approximately 76–77 °C while sustaining about 22.7 FPS over roughly 2,533 recorded inferences. No throttling or reboot was recorded, but the upward thermal trajectory means the run is evidence of short-duration stability only. The Raspberry Pi 5 is designed for substantially higher performance than earlier boards, and official documentation explicitly treats cooling as important for sustained workloads [16, 15]. A controlled active-cooling experiment is therefore a clear next step rather than a basis for claiming that the absence of a cooler caused any specific failure observed here.

The contrast between the 90-image temperatures and the continuous INT8 trace also illustrates why short benchmarks cannot establish thermal sustainability. INT8 finished the fixed workload near 55 °C, yet repeated inference continued to accumulate heat until the upper-70s within about two minutes. A format that finishes a short test at a moderate temperature could therefore behave very differently after several minutes. Long-run testing should record CPU frequency and throttle flags together with temperature so that performance degradation can be distinguished from a simple rise in sensor temperature.

4.5 Operational Stability and Runtime Failures

The negative runtime results are analytically important because a deployment format that cannot complete the intended workload cannot be ranked only from isolated inference speed. ONNX FP32 successfully processed a single image, proving that export and basic execution were possible, but sustained attempts ended in abrupt reboots. No valid 90-image aggregate ONNX benchmark therefore exists. Potential explanations include thermal, power, memory or runtime behaviour, but the logs do not isolate a

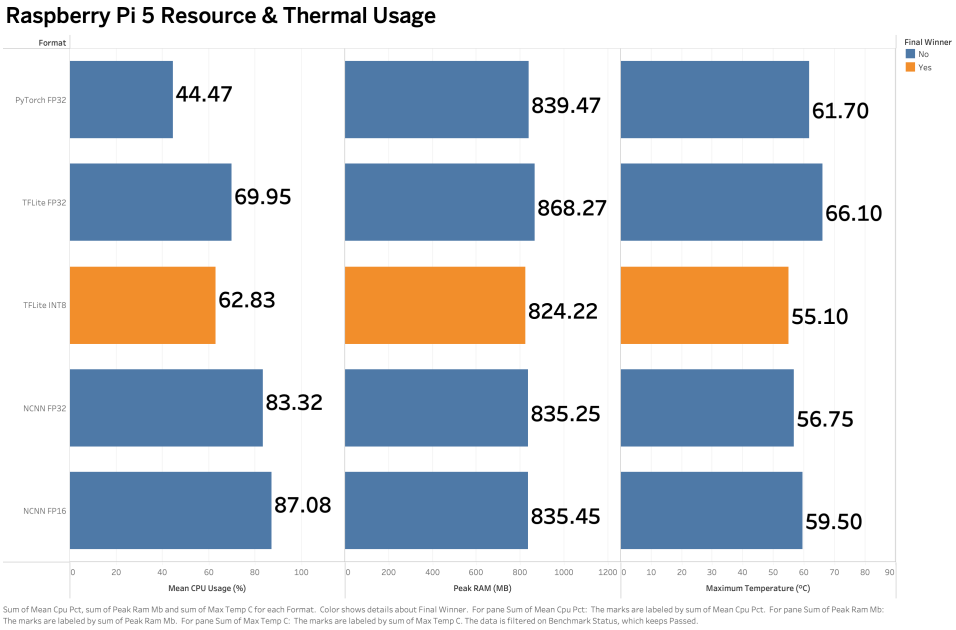


Figure 4.5: Mean CPU usage, peak process memory and maximum short-run temperature for each completed format. Faster inference does not translate uniformly into lower CPU load, memory demand or thermal output.

cause; assigning one would exceed the evidence.

NCNN FP32 presents a different lesson. It completed the fixed 90-image benchmark and therefore has valid aggregate metrics, yet a later continuous test hard-reset the Pi. Short benchmark completion is consequently weaker evidence than repeated sustained operation. TFLite INT8 is the only format for which both a complete 90-image benchmark and the genuine approximately two-minute continuous experiment are available.

The evidence hierarchy can therefore be stated explicitly. A successful export establishes file conversion; a successful single-image smoke test establishes basic functional compatibility; a complete 90-image benchmark establishes short-workload aggregate behaviour; and a continuous test adds evidence about sustained operation. These levels should not be collapsed. ONNX reached the single-image level, NCNN FP32 reached the fixed-benchmark level but failed a later continuous attempt, and INT8 reached the available continuous-test level. NCNN FP16 has strong fixed-benchmark metrics but was not subjected to equivalent continuous validation.

These outcomes support the general benchmarking principle that representative workloads and system-level measurements matter, rather than assuming portable model formats behave equivalently on a particular device [17]. They also strengthen the decision to report failures as results rather than substitute planned numbers or infer missing aggregate values.

4.6 Deployment Selection

For the tested Raspberry Pi 5 CPU-only configuration, TFLite INT8 provides the strongest combined evidence. It has the lowest mean latency (44.13 ms), highest median throughput (22.62 FPS), lowest peak process memory among the completed formats (824.22 MB), a successful 90-image run and the only completed continuous-inference test. The selection is therefore based on a combination of speed, resource profile and available stability evidence, not on speed alone.

The recommendation is explicitly contextual. TFLite INT8 has lower accuracy than the FP32 and FP16 alternatives, and a different hardware accelerator, cooling configuration, workload or accuracy requirement could alter the preferred point. NCNN FP16 is the strongest secondary candidate where accuracy retention is more important: it reached 13.82 FPS with mAP50 0.537, close to the PyTorch mAP50 of 0.548. Its limitation in this study is the absence of an equivalent completed continuous stability experiment.

A useful way to interpret the final choice is as a Pareto-style decision. PyTorch occupies the high-

accuracy/low-speed end; INT8 occupies the high-speed/lower-accuracy end; and NCNN FP16 lies between them. No single representation dominates the others on every criterion. The chosen point reflects the project's edge-computing emphasis and the evidence actually completed on the device. If the downstream system assigns a very high cost to missed rocks, the decision boundary could shift toward NCNN FP16 even before its throughput becomes the maximum.

The selection can be stress-tested against alternative priorities. If mAP50 retention were treated as the dominant objective, NCNN FP16 would be attractive because it loses only 0.011 absolute mAP50 from PyTorch while increasing throughput by more than fivefold. If maximum throughput and available continuous evidence dominate, INT8 is stronger. If simplicity and fidelity to the original training framework dominate, PyTorch avoids conversion but fails the speed objective. This exercise demonstrates that the recommendation is not a hidden weighted average chosen after seeing the results; it is a transparent prioritisation of edge responsiveness with an explicit accuracy constraint.

A production decision would require those priorities to be stated by the agricultural operator. For example, the cost of missing a rock may depend on machinery damage risk, while a false positive may cause an unnecessary stop or removal attempt. Such costs are not available in the current project, so the dissertation avoids converting mAP and count MAE into an unsupported economic utility score. The recommendation remains technical and configuration-specific.

The same caution applies to the apparent 22.62 FPS headline. Inference throughput is only one component of an eventual field pipeline. A production system would also need to decode camera frames, maintain timestamps, transform or associate coordinates, log detections and potentially communicate with a control system. Those operations consume CPU and memory that were not present in the stored-image benchmark. The measured value should therefore be read as available detector throughput under the test workload, not as a guaranteed end-to-end frame rate after all future components are integrated.

4.7 Threats to Validity and Relation to Existing Work

Several limitations bound the strength of the conclusions. First, the evaluation used a single Raspberry Pi 5 rather than repeated devices, so manufacturing variation and device-to-device thermal behaviour are not represented. Second, the executed benchmark used three warm-ups and one timed pass, not repeated independent timing runs. Tail latency and variance can therefore be reported from the pass but not as a robust distribution across repeated experiments.

Third, the 90-image test set was untouched for *architecture* selection, but was later reused to compare and choose the final edge runtime. There is no second untouched dataset that provides an unbiased final estimate after runtime selection. Fourth, the workload uses stored letterboxed images rather than a live camera pipeline. Image capture, decoding, frame buffering and field motion could change end-to-end latency. Fifth, the stability evidence is asymmetric: INT8 has a completed approximately two-minute continuous run, NCNN FP32 has a failed continuous attempt, and the other formats do not have equivalent long repeated tests.

Sixth, accuracy results depend on the selected confidence and IoU settings. Alternative operating points could change precision/recall and count error, especially for half-buried rocks. Finally, the project does not validate GPS mapping, size/weight estimation or robotic removal. It should be positioned alongside precision-agriculture edge research as a focused deployment experiment, not a field-ready autonomous system. This framing is consistent with reviews that identify Raspberry Pi as a useful low-cost agricultural platform while calling for broader domain and real-world validation [7], and with edge-agriculture literature that emphasises unresolved accuracy/resource trade-offs [20].

There is also a distinction between internal and external validity. Internal comparability is strengthened by fixing the images, detector and thresholds across completed runtimes. External validity is weaker because the results come from one board, one dataset domain and one stored-image pipeline. The study is therefore strongest when answering "which representation behaved best in this controlled configuration?" and weaker when answering "which representation will be best on all agricultural edge systems?" The latter claim is intentionally not made.

The choice of test set creates one further interpretive boundary. The 90 images were legitimately untouched when the detector architecture was selected, but after those same images were used to compare runtime representations and choose TFLite INT8, the final runtime decision became conditioned on them. Accordingly, the reported INT8 metrics describe performance on the selection workload, not a fresh post-selection estimate. A rigorous next experiment would freeze the complete detector-plus-runtime pipeline and evaluate it once on a second independently collected set. That step would test whether the measured INT8 compromise and the half-buried weakness persist outside the data used for the final edge choice.

The findings nevertheless align with several themes in related work. Integer quantisation has long been motivated as a way to enable efficient inference on resource-constrained processors [6]; the measured INT8 result demonstrates that the expected speed benefit can be substantial on the target Pi, while also showing a non-zero accuracy cost. MLPerf's emphasis on well-defined inference scenarios is reflected in the discovery that the same YOLOv8n detector spans roughly 2.5 to 22.6 FPS depending on representation [17]. In agricultural context, the persistence of a half-buried weakness is compatible with broader reviews that identify scene variability and dataset coverage as central challenges [1, 2].

The project also complements work using alternative sensing. Thürkow et al. investigate thermal imagery for agricultural stone detection [22], suggesting a potential route for cases in which RGB contrast is poor. The present study does not establish that thermal or depth sensing would solve the burial-status problem, but it provides a concrete reason to test such modalities: the dominant remaining weakness is semantic visibility of partially buried stones, not simply insufficient CPU throughput.

4.8 Overall Answer to the Research Question

The two-class YOLOv8n detector can be deployed effectively on a Raspberry Pi 5 for stored-image, CPU-only inference, but effectiveness depends strongly on representation. PyTorch FP32 provides the best measured aggregate detection quality among the completed edge runs but is slow at 397.48 ms mean latency and 2.52 median FPS. TFLite INT8 reduces mean latency to 44.13 ms and raises throughput to 22.62 FPS, while mAP50 decreases from 0.548 to 0.503 and mAP50–95 from 0.275 to 0.234. NCNN FP16 occupies an intermediate position at 13.82 FPS and mAP50 0.537.

The most important accuracy weakness is not solely a conversion artefact: half-buried rocks were already much harder than completely exposed rocks in the source detector. Operationally, five representations completed the fixed benchmark, whereas ONNX did not produce a valid aggregate run. TFLite INT8 also completed the available short continuous experiment without recorded throttling or reboot, although temperature rose to approximately 76–77 °C. The research question is therefore answered by a trade-off rather than a universal winner: TFLite INT8 is the preferred format for the tested proof-of-concept configuration, while NCNN FP16 remains a credible accuracy-retention alternative requiring stronger stability validation.

Chapter 5

Conclusion and Future Work

5.1 Overall Conclusion and Answer to the Research Question

This dissertation investigated how effectively a two-class YOLOv8n rock burial-status detector could be transferred from model development to Raspberry Pi 5 CPU inference. The project established a common 90-image edge workload, prepared six runtime representations and obtained complete aggregate results for five of them. The evidence shows that model representation has a substantial effect on practical deployment even when the trained detector is held constant.

PyTorch FP32 produced the strongest measured aggregate edge accuracy, with mAP50 0.548 and mAP50–95 0.275, but required 397.48 ms mean latency and achieved only 2.52 median FPS. TFLite INT8 reduced latency to 44.13 ms and increased throughput to 22.62 FPS—approximately a 9.01× latency speed-up—at the cost of reducing mAP50 to 0.503 and mAP50–95 to 0.234. NCNN FP16 offered a useful middle point with 13.82 FPS and mAP50 0.537. ONNX FP32 could process isolated images but did not complete a valid sustained aggregate benchmark.

For the tested CPU-only Raspberry Pi 5 configuration, TFLite INT8 is therefore the preferred representation because it combines the best measured latency/throughput, the lowest peak process memory among completed formats, a complete 90-image benchmark and the available short continuous stability evidence. This is not a claim that INT8 is universally superior. The appropriate runtime would change if accuracy requirements, hardware acceleration, cooling or field conditions changed.

5.2 Assessment of Project Objectives and Research Hypothesis

Table 5.1: Assessment of the project objectives against completed evidence.

Objective	Status
Establish the two-class detector and evaluation context	Achieved
Prepare six alternative edge-runtime representations	Achieved
Create a reproducible Raspberry Pi evaluation workflow	Achieved
Benchmark quality, latency, throughput and resource behaviour	Achieved for five formats; ONNX aggregate incomplete
Investigate operational reliability and negative results	Partially achieved; short/asymmetric stability evidence
Select and justify a preferred representation for the tested Pi	Achieved: TFLite INT8

The research hypothesis is supported. Every completed deployment-oriented representation reduced mean latency relative to PyTorch FP32, with speed-ups ranging from approximately 2.77× for TFLite FP32 to 9.01× for TFLite INT8. The magnitude of improvement varied substantially by runtime. The hypothesis also anticipated possible accuracy or robustness costs: INT8 showed the largest accuracy reduction, ONNX failed to complete sustained benchmarking, and NCNN FP32 failed a later continuous attempt despite completing the fixed benchmark. The evidence therefore supports the proposed accuracy–efficiency–robustness trade-off rather than a simple claim that lower precision is always better.

5.3 Current Project Status and Limitations

The resulting system is an academic edge-deployment proof of concept. It demonstrates stored-image rock detection and burial-status classification on a Raspberry Pi 5, not a complete autonomous field solution. Live-camera acquisition, GPS integration, verified rock size/weight estimation, 3D sensing and robotic removal remain outside the implemented scope described in the wider project materials [4, 3].

The strongest model-level limitation is half-buried recognition. The source detector already showed a large gap between completely exposed and half-buried rocks, so this issue should not be attributed solely to INT8 or FP16 conversion. Experimental validity is also constrained by the single Raspberry Pi, one timed pass per completed format, fixed thresholds, stored-image workload, short stability evidence and reuse of the 90-image set for final runtime selection after it had served as the untouched architecture-selection test. A second untouched deployment-validation set would be required for a cleaner final estimate of the selected runtime.

5.4 Prioritised Future Work

The first priority is to improve half-buried detection. Additional independently collected examples should cover different soil types, lighting, moisture, occlusion levels, rock textures and dense scenes. Hard-negative mining and targeted augmentation should be evaluated against a held-out field set rather than inferred from training behaviour alone.

Second, the runtime comparison should be repeated with multiple timed passes and longer stability experiments under controlled ambient conditions. Active cooling should be evaluated directly, with temperature, frequency/throttling state, power and latency recorded throughout. NCNN FP16 should receive the same continuous protocol as TFLite INT8 because its accuracy retention makes it a strong alternative.

Third, ONNX failure diagnosis should be made systematic. Repeated runs should isolate memory, power, thermal and runtime variables rather than inferring a cause from an abrupt reboot. Only a completed controlled run should be used to add ONNX to the aggregate ranking.

Fourth, evaluation should progress from stored images to a live-camera pipeline. End-to-end measurements should include frame acquisition, resizing/letterboxing, inference, post-processing and output handling. Once the detector is stable in that setting, the wider project can consider GPS association, rock-size estimation, multimodal or depth sensing, and eventual integration with robotic removal. These stages should be treated as new validation problems rather than assumed to follow automatically from successful detector inference.

5.5 Final Conclusion

The project demonstrates that a lightweight agricultural detector can run at useful speed on a Raspberry Pi 5, but deployment decisions materially change accuracy, latency, resource demand and operational confidence. TFLite INT8 provides the strongest overall evidence for the tested CPU-only proof of concept, while NCNN FP16 preserves more accuracy at lower throughput. The most important next challenge is not simply to make inference faster: it is to strengthen half-buried rock recognition and validate sustained, cooled, live-camera operation under realistic field conditions. Collectively, the contributions described here are outcomes of the group project; individual roles, decisions and learning are documented separately in the Individual Reflective Accounts.

Bibliography

[1] Mar Ariza-Sentís, Sergio Vélez, Raquel Martínez-Peña, Hilmy Baja, and João Valente. Object detection and tracking in precision farming: a systematic review. *Computers and Electronics in Agriculture*, 219:108757, 2024.

[2] Chetan M. Badgujar, Alwin Poulose, and Hao Gan. Agricultural object detection with you only look once (YOLO) algorithm: A bibliometric and systematic literature review. *Computers and Electronics in Agriculture*, 223:109090, 2024.

[3] Felipe Campelo. On-the-go detection of field rocks for precision agriculture in potato fields: project kickoff slides, 2026. University of Bristol group-project meeting material, unpublished.

[4] Dalhousie University and McCain Foods. On-the-go detection of field rocks for precision agriculture in potato fields: project flyer, 2026. Unpublished project material supplied to the authors.

[5] Google. LiteRT documentation, 2026. Accessed 23 August 2026.

[6] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018.

[7] Astina Joice, Talha Tufaique, Humeera Tazeen, C. Igathinathane, Zhao Zhang, Craig Whippo, John Hendrickson, and David Archer. Applications of raspberry pi for precision agriculture—a systematic review. *Agriculture*, 15(3):227, 2025.

[8] Yogeswaranathan Kalyani and Rem Collier. A systematic survey on the role of cloud, fog, and edge computing combination in smart agriculture. *Sensors*, 21(17):5922, 2021.

[9] Xiang Li, Wenhai Wang, Lijun Wu, Shuo Chen, Xiaolin Hu, Jun Li, Jinhui Tang, and Jian Yang. Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In *Advances in Neural Information Processing Systems*, volume 33, pages 21002–21012, 2020.

[10] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2117–2125, 2017.

[11] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *Computer Vision—ECCV 2014*, volume 8693 of *Lecture Notes in Computer Science*, pages 740–755. Springer, 2014.

[12] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. SSD: Single shot multibox detector. In *Computer Vision—ECCV 2016*, volume 9905 of *Lecture Notes in Computer Science*, pages 21–37. Springer, 2016.

[13] Microsoft. ONNX Runtime documentation, 2026. Accessed 23 August 2026.

[14] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763, 2021.

[15] Raspberry Pi Ltd. Cooling raspberry pi 5, 2026. Accessed 23 August 2026.

[16] Raspberry Pi Ltd. Raspberry pi 5 documentation, 2026. Accessed 23 August 2026.

[17] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, et al. MLPerf inference benchmark. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture*, pages 446–459, 2020.

[18] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016.

[19] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, volume 28, pages 91–99, 2015.

[20] Sapna, Himanshu Gauttam, Vikas Chauhan, Kiran Kumar Pattanaik, Aditya Trivedi, and Hrishita Ghosh. A comprehensive review of edge computing empowered smart agriculture: Trends, opportunities and future directions. *Computers and Electronics in Agriculture*, 241:111252, 2026.

[21] Tencent. NCNN: a high-performance neural-network inference computing framework optimised for mobile platforms, 2026. Accessed 23 August 2026.

[22] Florian Thürkow, Mike Teucher, Detlef Thürkow, and Milena Mohri. Stone detection on agricultural land using thermal imagery from unmanned aerial systems. *AgriEngineering*, 7(7):203, 2025.

[23] Ultralytics. Ultralytics YOLO model documentation, 2026. Accessed 23 August 2026.

[24] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems*, volume 27, pages 3320–3328, 2014.

Appendix A

Dataset and Detector Development Evidence

The original dataset contained 448 images and 706 labelled rocks: 362 completely exposed and 344 half buried. The pre-augmentation training/validation/test split is reported in Table 3.1. Training-only augmentation generated 1,878 additional images, producing 2,191 training images and 3,563 annotation instances across the augmented copies.

Table A.1: Extended detector-development evidence.

Model	Precision	Recall	F1	mAP50	mAP50–95
YOLOv8n	0.681	0.759	0.718	0.759	0.4288
YOLO11n	0.743	0.719	0.731	0.729	0.4002
YOLO11s	0.764	0.801	0.782	0.783	0.4107

The selected YOLOv8n produced untouched-test mAP50–95 of approximately 0.3384. At the evaluated source-detector operating point, the class-specific confusion result represented 64/69 completely exposed rocks correctly and 23/66 half-buried rocks correctly. These values are operating-point correct rates and must not be relabelled as AP.

Appendix B

Raspberry Pi Deployment and Reproducibility

The six prepared representations were PyTorch FP32, ONNX FP32, TFLite FP32, TFLite INT8, NCNN FP32 and NCNN FP16. Final inference used fixed 640×640 letterboxed images with padding value 114, batch size one, confidence threshold 0.25 and IoU setting 0.70. Three warm-up inferences preceded one timed 90-image pass. The ten-image smoke test was a subset of the same 90 images and served only as a functional check.

INT8 calibration used 200 original training images containing 304 labelled objects, balanced at 152 completely exposed and 152 half buried. The Raspberry Pi environment was Debian GNU/Linux 13, Python 3.13.5, Ultralytics 8.4.115, ONNX Runtime 1.28.0, OpenCV 5.0.0, NumPy 2.5.1, Pandas 3.0.5 and psutil 7.2.2. The device used the official 27 W 5.1 V/5 A supply and no Active Cooler. No camera was used in the benchmark.

Appendix C

Extended Raspberry Pi Results

Table C.1: Extended canonical edge metrics used in the main evaluation.

Format	Latency ms	FPS	CPU %	RAM MB	P	R	F1	mAP50	mAP50–95	Count MAE
PyTorch FP32	397.48	2.52	44.47	839.47	0.716	0.550	0.622	0.548	0.275	0.600
TFLite FP32	143.70	6.95	69.95	868.27	0.669	0.542	0.599	0.535	0.270	0.600
TFLite INT8	44.13	22.62	62.83	824.22	0.661	0.539	0.594	0.503	0.234	0.633
NCNN FP32	75.09	13.26	83.32	835.25	0.667	0.542	0.598	0.535	0.271	0.600
NCNN FP16	72.31	13.82	87.08	835.45	0.672	0.542	0.600	0.537	0.270	0.611
ONNX FP32	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

Approximate maximum temperatures during the fixed benchmark were 55.10, 56.75, 59.50, 61.70 and 66.10 °C for TFLite INT8, NCNN FP32, NCNN FP16, PyTorch FP32 and TFLite FP32 respectively. These values are short-workload observations and not long-run thermal equilibria.

Appendix D

Stability and Failure Diagnostics

TFLite INT8 completed the only valid continuous-inference experiment retained for final analysis: approximately two minutes, roughly 2,533 inference rows and about 22.7 FPS. Temperature rose from the mid-40s to about 76–77 °C (raw maximum approximately 77.1 °C), with no recorded reboot or throttling during the run.

ONNX FP32 processed isolated images but did not complete a valid 90-image aggregate benchmark; sustained attempts caused abrupt Raspberry Pi reboots and the root cause remains unresolved. NCNN FP32 completed the 90-image benchmark but a later continuous attempt hard-reset the device and did not yield a valid completed continuous CSV. No genuine 10-minute or 30-minute final stability test was completed. A file whose name suggested a 30-minute test duplicated the genuine two-minute data and is excluded from the evidence base.